Інструкція до практичної роботи №79
ТЕМА: "Створення програм з функціями."

МЕТА: Ознайомитись з функціями мові С++. Набути практичних навичок створення програм з використанням функцій.

Теоретичні відомості

Структурний аналіз полягає у попередньому опрацювання складної задачі чи громіздкого алгоритму та поділ його на окремі простіші частини.

У С++ ці частини реалізуються за допомогою функцій. Окремі функції об'єднуються у спільну програму. У відкомпільованому вигляді така програма утворює модуль.

Головна функція, що обов'язково входить до кожної програми – main ().

Стандартні функції описані у бібліотеках.

Функція користувача – це поіменована група команд, яка оголошена у файлі заголовків (або в основній програмі) та описана у модулі (в основній програмі). До функції можна звертатися (викликати) з будь-якого місця програми необхідну кількість разів. Зазвичай функцій описують на початку програми, але можна порушувати таку закономірність. Тоді рекомендується оголошувати функції

Опис функцій користувача:
Тип_функції назва_функції (список_формальних параметрів)
{
тіло_функцій;
return (змінна);
}
Назву функцій надає користувач за правилом створення ідентифікаторів
Тип функцій – це тип змінної, яку функція повертає в основну програму.

Виклик функції
Назва_функції (список_фактичних параметрів);

До функції звертаються з розділу команд основної функції або з іншої функції.

Виклик можна виконувати або командою виклику, або з виразів.

Список фактичних параметрів може містити сталі, змінні, посилання, вказівники, вирази.

Списки формальних та фактичних параметрів мають бути узгодженими за типами та кількістю елементів. **Увага!** Якщо у списку формальних параметрів є проініціалізовані змінні, то у списку фактичних параметрів змінні можуть бути відсутні, їх будуть надані значення за замовчуванням

Приклади виклику функцій
Нехай заголовок функції
float perimetr (int k=4, float r=2.5)
До цієї функції можна звернутися одним із способів:

perimetr (7, 2.8); k=7, r=2.8
perimetr (8); k=8, r=2.5
perimetr (); k=4, r=2.5

Глобальні та локальні змінні. Область видимості

Область видимості змінної – це частина програми, де змінна оголошена або, де до неї можна отримати доступ за допомогою операції надання видимості (::).

Локальна змінна являє собою змінну, оголошену всередині функції до її використання. Область її видимості: від точки оголошення до кінця даної функції.

Увага! Імена змінних у різних функціях можуть збігатися, але це не призводить до конфлікту. С++ трактує ім'я кожної змінної як локальне стосовно відповідної функції.

Глобальні змінні – оголошуються поза усіма функціями (зокрема, поза функцією main). Область їх видимості – від точки оголошення до кінця програми (глобально для всіх функцій).

Увага! *Якщо ваша програма використовує глобальну змінну і її ім'я співпадає з ім'ям локальної змінної, то С++ надає пріоритет локальній змінної. Як правило, у програмах варто уникати використання глобальних змінних.*

Операція видимості. Для того, щоб звернутися до глобальної змінної, чиє ім’я конфліктує з ім’ям локальної змінної, використовують операцію надання видимості. Наприклад, ::num=2;

Випадкові числа

Заголовковий файл **<stdlib.h>** містить функцію **random(x),** яка повертає випадкове ціле число в діапазоні від 0 до x. Перед першим звертанням до функції random(x) рекомендується викликати функцію **randomize().** Якщо цього не зробити, то при кожному запуску програми будемо мати однакову послідовність випадкових чисел.

Приклад 3.

Використовуючи перевантаження функцій, скласти програму для упорядкування трьох уведених даних або символьного, або цілого типу.

```
#include<iostream.h>
#include<conio.h>
void sort(int n1, int n2, int n3);
        void sort(char n1, char n2, char n3);
                                                    int main()
{int i1, i2, i3,k;
         char c1, c2, c3;
                  cout<<“uvedit 1 dlya simvoliv, 2 – dlya chisel\n”; cin>>k;
if (k==1)
         { cout<<“3 simvoli: “;
           cin>>c1>>c2>>c3;
           sort( c1, c2, c3); }
```

```
    if (k==2)
    { cout<<"3 chisla: ";
      cin>>i1>>i2>>i3;
sort( i1, i2, i3);}
 getch ();return 0;}
//-----------------------------------------
void sort(int n1, int n2, int n3)
{int c;
 if (n1>n2) {c=n1; n1=n2; n2=c;}
 if (n2>n3) {c=n2; n2=n3; n3=c;}
 if (n1>n2) {c=n1; n1=n2; n2=c;}
 cout<<n1<<"\t"<<n2<<"\t|<<n3;}
//-----------------------------------------
void sort(char n1, char n2, char n3)
{char c;
 if (n1>n2) {c=n1; n1=n2; n2=c;}
 if (n2>n3) {c=n2; n2=n3; n3=c;}
 if (n1>n2) {c=n1; n1=n2; n2=c;}
 cout<<n1<<"\t"<<n2<<"\t|<<n3;}
```

Шаблони функцій. Основні поняття

Перевантаження функцій часто буває корисним, але не завжди зручним. Як видно з попереднього прикладу, один і той самий код довелося записувати окремо для змінних символьного типу, а потім для цілих чисел. У мові С++ є змога уникнути таке дублювання за допомогою шаблонів.

Шаблон функції – це опис функції, яка не залежить від даних певного типу. Крім того, така функція може повертати в основну програму результат довільного типу.

Під час виклику такої функції компілятор автоматично проаналізує тип фактичних аргументів, згенерує для них програмний код і виконає відповідні дії.

Такий підхід у програмуванні називається **узагальненим програмуванням**.

Приклад 4.

Використовуючи шаблони функцій, скласти програму для упорядкування трьох уведених даних або символьного, або цілого типу.

```
                                                    #include<iostream.h>
#include<conio.h>
// оголошення шаблона функції
        template <class myType>
       void sort(myType n1, myType n2, myType n3);
                                                     int          main()
{
        int i1, i2, i3,k;
```

```
        char c1, c2, c3;
                cout<<“uvedit 1 dlya simvoliv, 2 – dlya chisel\n”; cin>>k;
if (k==1)
        { cout<<“3 simvoli: “;
          cin>>c1>>c2>>c3;
          sort( c1, c2, c3); } // виклик функції

      if (k==2)
        { cout<<“3 chisla: “;
          cin>>i1>>i2>>i3;
          sort( i1, i2, i3);} // виклик функції
       getch ();return 0;
      }
      //-----------------------------------------
      // опис шаблона функцій
      template <class myType>
      void sort(myType n1, myType n2, myType n3);
      {
       myType c;
       if (n1>n2) {c=n1; n1=n2; n2=c;}
       if (n2>n3) {c=n2; n2=n3; n3=c;}
       if (n1>n2) {c=n1; n1=n2; n2=c;}
       cout<<n1<<“\t”<<n2<<“\t|<<n3;
       }
```

Увага! У стандарті ISO/ANSI замість ключового слова class прийнято записувати typename

Вбудовані функції. Основні поняття

Під час виконання програми генеруються коди для всіх функцій користувача. Коли відбувається звернення до тієї чи іншої функції, здійснюється виклик відповідного коду. Якщо до функції звертаються багато разів, то робота програми сповільнюється.

Щоб цього уникнути, можна використовувати вбудовані функції. Коди таких функцій вставляються компілятором у кожну точку виклику в основному коді. Це вимагає додаткового обсягу оперативної пам'яті, проте прискорює процес виконання програми.

Щоб створити вбудовану функцію, потрібно до її оголошення додати ключове слово inline

Приклад 5.

Використовуючи вбудовані функції, скласти програму для обчислення добутку

$$\prod_{k=1}^{10} \frac{5k+2}{k-15}$$

```
#include<iostream.h>
#include<conio.h>
// оголошення вбудованої функції
      inline float dob( int a);
     void main()
{
      int n=10; float D=1;
      for (int k=1; k<=n; k++)
      D=dob(k);
      cout<<"D= "<<D;
      getch ();
     }
//--------------------------------------
float dob(int a)
 {
 static float d=1.0;
  d=d*(5*a+2)/(a-15);
  return d;
}
```

Результат виконання 28380.5

Запитання: Чи прискорює роботу наведеної програми використання вбудованої функції?

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Дані для перевірки залежать від N (mod — остача від ділення):

Завдання	**Дані варіанта**	**Приклад N = 7**
2А	цілі: N, N + 2, N + 4, N + 6, N + 8; дійсні: N + 0,5; N + 2; N + 4,25; N + 6; N + 8	[7, 9, 11, 13, 15]; [7.5, 9, 11.25, 13, 15]
2Б	трійка (N; 3N mod 13; N + 4)	(7; 8; 11)

б) Кожне завдання — окремий файл Прізвище_ПР79_зN.cpp (наприклад, Ivanenko_PR79_z2.cpp), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Компілюйте й запускайте програми в Code::Blocks, Dev-C++, Visual Studio чи іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот вікна виконання кожної програми з введеними даними.

г) Підключайте <iostream> (за потреби <cmath>, <iomanip>) і using namespace std; заголовки <iostream.h> і <conio.h> із прикладів застаріли — замість getch() використовуйте cin.get() або system("pause"). Якщо кирилиця в консолі спотворюється, додайте SetConsoleCP(1251); SetConsoleOutputCP(1251); (<windows.h>). Тексти виводу — точно як у прикладах. Перевантажені функції мають однакову назву й різні типи параметрів; не копіюйте тіло функції через умовні оператори на тип.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

Хід роботи

1. Перевірте роботу програм наведених в прикладах 3-5.

2. Виконайте завдання:

А). Використовуючи перевантаження функцій, складіть програму для обчислення суми п'яти введених даних. Дані можуть бути цілого або дійсного типу.

Формат: спочатку введіть тип даних («1» — цілі, «2» — дійсні), потім п'ять чисел; виведіть «Сума = …».

Приклад (N = 7): 1 і 7 9 11 13 15 → «Сума = 55»; 2 і 7.5 9 11.25 13 15 → «Сума = 55.75»; 1 і 1 2 3 4 5 → «Сума = 15».

Б). Написати функцію, що повертає більше з двох цілих чисел. Використовуючи цю функцію, написати програму для знаходження більшого з трьох чисел

Приклад (N = 7): 7 8 11 → «Більше: 11»; 5 5 5 → «Більше: 5»; -3 -1 -2 → «Більше: -1». Функцію max() із бібліотеки використовувати не можна.

Програми і результат їх роботи записати в зошит.

Критерії оцінювання

Складова	**Оцінювання**	**Макс. балів**
Завдання 1	2 бали	2
Завдання 2А	4,5 бала	4.5
Завдання 2Б	4,5 бала	4.5
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (не виконано вимогу умови, не використано перевантаження, неправильний формат результату) — 50%; програма не працює або не здана — 0. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити кількість чисел чи тип параметрів). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

1. Що таке функція в C++ і для чого її використовують?

2. Що таке перевантаження функцій і чим розрізняються перевантажені функції?
3. Що таке прототип функції?
4. Як передати в функцію кілька параметрів різних типів?

Відповідайте на контрольні питання письмово у звіті (документ Word разом зі скріншотами): 1–2 речення на кожне питання, власними словами.

Література

1. Глинський Я.М., Анохін В.Є., Ряжська В.А. C++ і C++ Builder. Навчальний посібник. – Львів: СПД Глинський, 2006.
2. Глушаков С.В., Смирнов С.В., Коваль А.В. Практикум по C++. – Харьков: Фоліо, 2006.
3. Лєхан С.Ф. Інформатика C++. Спецкурс. 10-12 класи. – Шепетівка; Аспект, 2007.